General Information

Welcome to Project Eagle! For the duration of our project we have decided to prepare a string of tasks for you to solve. The tasks and their deadlines will all be released in accordance to a carefully designed schedule at different times of the year.

Each task will focus on one main goal which is to be achieved by following the outlined steps in the given documents. At the end of each task you will be asked to **submit a working function with predetermined parameters and a GitHub wiki describing the inner workings of your function file**. The entirety of your code will have to be written in C++.

This project also has the secondary goal of teaching proper GitHub usage. Therefore, all of your solutions should be submitted directly to a repository, and your documentation should be placed in a linked GitHub Wiki page. The Tasks as well as any required material will be provided by us onto our own GitHub. This will include the tasks and possibly some helpful sources fitting to the tasks nature.

If while solving the tasks any questions or the need for help arises contact any one of us, we will try to answer as quickly as possible. We are excited to see your solutions and wish you all the best of luck in solving these following tasks!

Task 1 – Detecting the Markers

To better understand the thought process behind the first two tasks, you will have to understand our special use for ArUco-Markers.

ArUco-Markers similar to a simplified QR code. What's special about these markers is that they come with predefined dictionaries and a generation algorithm. Each dictionary contains a set of mathematically defined marker patterns, and with different IDs one can automatically generate an ArUco-Marker of any size that is unique within its dictionary. During detection, the extracted marker pattern from an image is compared against the dictionary's valid patterns to determine the corresponding ID.

Moreover, because of the unique shape of each ArUco-Marker and the fact that its ideal "digital twin" can be generated from the dictionary at any time, any ArUco-Marker contained in a picture can be isolated and compared to its generated version. This, in turn, **allows us to determine the marker's exact position and orientation relative to the camera**. What this means is that by running a loop of a self-written detection algorithm we can determine the id and the coordinates of any ArUco-Marker contained in a live video feed.

This is the fundamental base of Project Eagle.

In Task 1 you will be asked to detect each marker's ID and outline their existence. Later in Task 2 you will be programming the code to detect each marker's position. The idea is that the function for task 1 returns the edited video feed for task 2 to analyze.

Task 1

Task 1 will be centered around the creation of a working ArUco-marker detection algorithm. For this you will use the provided webcam as well as the OpenCV library. This Task will be separated into two main subtasks:

1. Calibrating the camera within the OpenCV library
2. Writing the actual detection using the OpenCV library

For Subtask 1, the solution will require you to:

1. Create a program that allows you to take pictures with the OpenCV library.
2. Create a function that analyses made pictures and returns a camera specific calibration matrix and distance coefficients.

With this calibration matrix you can now consistently use the provided webcam within the opencv library for expected results. This allows us to move on to Subtask 2.

THIS MAY WELL BE THE MOST IMPORTANT STEP OF THE PROJECT. GETTING THIS WRONG WILL RESULT IN FOLLOWING TASKS NOT WORKING.

For Subtask 2 it's all about actually tackling the essence of this first task. The end goal is to display the live video feed of the camera on your computer with outlines of any existing ArUco-markers and their ID. For this task, your file structure will consist of main.cpp and outline_detect.cpp. The main.cpp file will serve as the core of your code for the next few tasks, so we recommend keeping it clean. **The main function should contain an infinite loop that passes the video frames to the required processing functions and displays the resulting video feed.** It should also feature a simple way to stop the analysis and video feed by pressing a specific key.

Any additional features required for this and future tasks must be implemented only inside their designated task function, NOT IN MAIN.

Prerequisites for this task's solution will be:

- An understandable calibration code
- A cleam main.cpp structure
- A working video loop inside your main
- A pop-up video feed
- An overlay of colored outlines around detected markers
- An overlay of displayed IDs next to detected markers
- The capability for this to work with any number of markers
- An integrated way to stop the feed and detection
- A detailed wiki describing your progress

This Task will require you to submit a working function called outline_detect and a center file structure titled main. The functions form is dictated to be:

```
(*ids[],*corners[])=outline_detect(frame,calibration_matrix,distance_coefficients);
```